Structure in C language

28 October 2026 11:14

A **structure** in C is a user defined data type that allows you to combine multiple variables of different data types under a single name.

Structures are useful when you want to represent a real world entities such as a student, employee, book, product, or bank account whose information consists of several different types of data.

Why do we need structures in C?

Suppose you want to store the details of a student:

- Roll number - int
- Name - string(character array)
- Marks - floating point
- Grade - character(char)

Without a structure you would declare separate variables:

```
int rollNo = 109;
char name[30] = "Anchit";
float marks = 87.7;
char grade = 'A';
```

This works for one student, but managing the details of hundreds of students becomes difficult. The variables are separate and are not logically grouped.

A structure solves this problem by grouping a related data into one unit.

Syntax of a structure

The general syntax:

```
struct structureName{
   //structure variables
   Data-type member1;
   Data-type member2;
   Data-type member3;
   Data-type member4;
};
```

Explanation:

- Strut is the C keyword used to declare a structure type.
- structureName is the name you give to the structure.
- member1, member2 etc., are the individual variables inside it called structure member variables or fields.

- The ';' (semicolon) after the closing brace is mandatory.

```c
#include<stdio.h>
struct Student{
    int rollNo;
    char name[30];
    float marks;
    char grade;
};
```

This defines a structure type called struct Student. It describes what information a student record will contain.

Declaring structure variables.
There are two common way to declare structure variable:

Method one: Declare variable after defining the structure. (mostly used)

```c
#include<stdio.h>
struct Student{
    int rollNo;
    char name[30];
    float marks;
    char grade;
};

int main(int argc, char const *argv[])
{
    struct Student s1;
    struct Student s2;
    return 0;
}
```

Method 2: Declare variables immediately after the definition.

```c
#include<stdio.h>
struct Student{
    int rollNo;
    char name[30];
    float marks;
    char grade;
} s1, s2;
```

This defines the structure type and declare 2 variables s1 and s2 in the same statement.

Accessing structure member using dot(.) operator.

Structrevariable.memberName;

```c
#include<stdio.h>
#include<string.h>
struct Student{
    int rollNo;
    char name[30];
    float marks;
    char grade;
};
int main(int argc, char const *argv[])
{
    struct Student s1;
    struct Student s2;
    s1.rollNo = 101;
    strcpy(s1.name, "Ajay");
    s1.marks = 89.6f;
    s1.grade = 'A';

    s2.rollNo = 102;
    strcpy(s2.name, "Vijay");
    s2.marks = 79.6f;
    s2.grade = 'B';
    printf("\nStudent Information\n");
    printf("Roll number: %d\n", s1.rollNo);
    printf("Name: %s\n", s1.name);
    printf("Marks: %f\n", s1.marks);
    printf("Grade: %c\n", s1.grade);
    printf("\nStudent Information\n");
    printf("Roll number: %d\n", s2.rollNo);
    printf("Name: %s\n", s2.name);
    printf("Marks: %f\n", s2.marks);
    printf("Grade: %c\n", s2.grade);
    return 0;
}
```

Output:

```
Student Information
Roll number: 101
```

```
Name: Ajay
Marks: 89.599998
Grade: A

Student Information
Roll number: 102
Name: Vijay
Marks: 79.599998
Grade: B
```

Initializing a structure

```c
#include<stdio.h>
#include<string.h>
struct Student{
    int rollNo;
    char name[30];
    float marks;
    char grade;
};
int main(int argc, char const *argv[])
{
    struct Student s1;
    struct Student s2;
    s1.rollNo = 101;
    strcpy(s1.name, "Ajay");
    s1.marks = 89.6f;
    s1.grade = 'A';

    s2.rollNo = 102;
    strcpy(s2.name, "Vijay");
    s2.marks = 79.6f;
    s2.grade = 'B';
    printf("\nStudent Information\n");
    printf("Roll number: %d\n", s1.rollNo);
    printf("Name: %s\n", s1.name);
    printf("Marks: %f\n", s1.marks);
    printf("Grade: %c\n", s1.grade);
    printf("\nStudent Information\n");
    printf("Roll number: %d\n", s2.rollNo);
    printf("Name: %s\n", s2.name);
    printf("Marks: %f\n", s2.marks);
    printf("Grade: %c\n", s2.grade);
     //Initializing a structure
    struct Student s3 = {103, "Rahul", 81.5, 'B'};
```

```
    printf("\nStudent Information\n");
    printf("Roll number: %d\n", s3.rollNo);
    printf("Name: %s\n", s3.name);
    printf("Marks: %f\n", s3.marks);
    printf("Grade: %c\n", s3.grade);
    return 0;
}

Output:
   Student Information
   Roll number: 101
   Name: Ajay
   Marks: 89.599998
   Grade: A

   Student Information
   Roll number: 102
   Name: Vijay
   Marks: 79.599998
   Grade: B

   Student Information
   Roll number: 103
   Name: Rahul
   Marks: 81.500000
   Grade: B



   #include<stdio.h>
   #include<string.h>
   struct Student{
       int rollNo;
       char name[30];
       float marks;
       char grade;
   };
   int main(int argc, char const *argv[])
   {
       struct Student s1;
       struct Student s2;
       s1.rollNo = 101;
       strcpy(s1.name, "Ajay");
       s1.marks = 89.6f;
       s1.grade = 'A';

       s2.rollNo = 102;
```

```
    strcpy(s2.name, "Vijay");
    s2.marks = 79.6f;
    s2.grade = 'B';
    printf("\nStudent Information\n");
    printf("Roll number: %d\n", s1.rollNo);
    printf("Name: %s\n", s1.name);
    printf("Marks: %f\n", s1.marks);
    printf("Grade: %c\n", s1.grade);
    printf("\nStudent Information\n");
    printf("Roll number: %d\n", s2.rollNo);
    printf("Name: %s\n", s2.name);
    printf("Marks: %f\n", s2.marks);
    printf("Grade: %c\n", s2.grade);
    struct Student s3 = {103, "Rahul", 81.5, 'B'};
    printf("\nStudent Information\n");
    printf("Roll number: %d\n", s3.rollNo);
    printf("Name: %s\n", s3.name);
    printf("Marks: %f\n", s3.marks);
    printf("Grade: %c\n", s3.grade);
    struct Student s4;
    printf("Enter roll number: ");
    scanf("%d", &s4.rollNo);
    getchar();
    printf("Enter name: ");
    fgets(s4.name, 30, stdin);
    printf("Enter marks: ");
    scanf("%f", &s4.marks);
    getchar();
    printf("Enter grade: ");
    scanf("%c", &s4.grade);
    printf("\nStudent Information\n");
    printf("Roll number: %d\n", s4.rollNo);
    printf("Name: %s", s4.name);
    printf("Marks: %f\n", s4.marks);
    printf("Grade: %c\n", s4.grade);
    return 0;
}
```

```
Output:
   Student Information
   Roll number: 101
   Name: Ajay
   Marks: 89.599998
   Grade: A

   Student Information
   Roll number: 102
```

```
Name: Vijay
Marks: 79.599998
Grade: B

Student Information
Roll number: 103
Name: Rahul
Marks: 81.500000
Grade: B
Enter roll number: 104
Enter name: Harsh
Enter marks: 92.3
Enter grade: A

Student Information
Roll number: 104
Name: Harsh
Marks: 92.300003
Grade: A
```

Arrays of Structure

An array of structure stores multiple records of the same structure type. Suppose you want to store the details of five students. Instead of declaring five separate variables (S1,S2,S3 and so on) you can declare an array.

```c
#include<stdio.h>

#define SIZE 3
struct Student{
    int rollNo;
    char name[30];
    float marks;
    char grade;
};
int main(int argc, char const *argv[])
{
    struct Student sArr[SIZE];
    int i;
    for(i = 0; i < SIZE; i++){
        printf("Enter roll number: ");
        scanf("%d", &sArr[i].rollNo);
        getchar();
        printf("Enter name: ");
        fgets(sArr[i].name, 30, stdin);
        printf("Enter marks: ");
```

```
        scanf("%f", &sArr[i].marks);
        getchar();
        printf("Enter grade: ");
        scanf("%c", &sArr[i].grade);
    }
    printf("\nStudent Information\n");
    for(i = 0; i < SIZE; i++){
        printf("Roll number: %d\n", sArr[i].rollNo);
        printf("Name: %s", sArr[i].name);
        printf("Marks: %f\n", sArr[i].marks);
        printf("Grade: %c\n", sArr[i].grade);
    }
    return 0;
}
```

Output:

```
   Enter roll number: 101
   Enter name: Shiv
   Enter marks: 99.8
   Enter grade: A
   Enter roll number: 102
   Enter name: Ganesh
   Enter marks: 98.9
   Enter grade: A
   Enter roll number: 103
   Enter name: Kartik
   Enter marks: 88.9
   Enter grade: B

   Student Information
   Roll number: 101
   Name: Shiv
   Marks: 99.800003
   Grade: A
   Roll number: 102
   Name: Ganesh
   Marks: 98.900002
   Grade: A
   Roll number: 103
   Name: Kartik
   Marks: 88.900002
   Grade: B
```

Nested structures

A nested structure is a structure that contains another structure as one of its members. For example, a student may have a date of birth consisting of a day, month and year.

```c
#include<stdio.h>

#define SIZE 3
struct Date
{
    int day, month, year;
};
struct Student{
    int rollNo;
    char name[30];
    float marks;
    char grade;
    struct Date dob;
};
int main(int argc, char const *argv[])
{
    struct Student sArr[SIZE];
    int i;
    for(i = 0; i < SIZE; i++){
        printf("Enter roll number: ");
        scanf("%d", &sArr[i].rollNo);
        getchar();
        printf("Enter name: ");
        fgets(sArr[i].name, 30, stdin);
        printf("Enter marks: ");
        scanf("%f", &sArr[i].marks);
        getchar();
        printf("Enter grade: ");
        scanf("%c", &sArr[i].grade);
        printf("Enter date of birth\n");
        printf("Enter Day: ");
        scanf("%d", &sArr[i].dob.day);
        printf("Enter Month: ");
        scanf("%d", &sArr[i].dob.month);
        printf("Enter Year: ");
        scanf("%d", &sArr[i].dob.year);
    }
    printf("\nStudent Information\n");
    for(i = 0; i < SIZE; i++){
        printf("Roll number: %d\n", sArr[i].rollNo);
        printf("Name: %s", sArr[i].name);
        printf("Marks: %f\n", sArr[i].marks);
        printf("Grade: %c\n", sArr[i].grade);
        printf("Date of birth: %d-%d-%d\n", sArr[i].dob.day,
sArr[i].dob.month, sArr[i].dob.year);
```

```
    }
    return 0;
}
```

```
Output:
   Enter roll number: 101
   Enter name: Gagan
   Enter marks: 60
   Enter grade: D
   Enter date of birth
   Enter Day: 12
   Enter Month: 11
   Enter Year: 2007
   Enter roll number: 102
   Enter name: Naman
   Enter marks: 77
   Enter grade: C
   Enter date of birth
   Enter Day: 13
   Enter Month: 2
   Enter Year: 2007
   Enter roll number: 103
   Enter name: Karan
   Enter marks: 89.9
   Enter grade: A
   Enter date of birth
   Enter Day: 5
   Enter Month: 5
   Enter Year: 2007

   Student Information
   Roll number: 101
   Name: Gagan
   Marks: 60.000000
   Grade: D
   Date of birth: 12-11-2007
   Roll number: 102
   Name: Naman
   Marks: 77.000000
   Grade: C
   Date of birth: 13-2-2007
   Roll number: 103
   Name: Karan
   Marks: 89.900002
   Grade: A
   Date of birth: 5-5-2007
```